PROJECT REVIEW 3

IMPLEMENTATION DOCUMENTATION TEMPLATE

REVIEW 3 REQUIREMENT: 100% IMPLEMENTATION COMPLETED

Project Title	IntelliCannula: Two-Module Clinical Decision Support for Peripheral IV Cannulation (Pre-Cannulation Vein Guidance & Post-Cannulation Infection Monitoring)
Student Name(s)	Sastika M, Akshaya GK
Registration Number(s)	23MIA1047, 23MIA1127
Programme / Branch	M.Tech. Integrated Computer Science and Engineering (Business Analytics)
Course / Project Code	SWE1017 — Project Review 3 (Final Implementation Review)
Faculty Guide	Dr. Manjula V. (Professor, School of Computer Science and Engineering)
Review	Review 3 - Final Implementation Review

Review 3 Submission Checklist

	Item	Status
☑	Complete source code submitted	*Yes*
☑	All modules integrated and executable	*Yes*
☑	Dataset / database / sample input included or access provided	*Yes*
☑	Dependencies / requirements file included	*Yes*
☑	README / execution instructions included	*Yes*
☑	Screenshots and outputs included	*Yes*
☑	Testing and validation completed	*Yes*
☑	Results / performance metrics reported	*Yes*
☑	Documentation completed	*Yes*

Submission notes: the trained checkpoints (`vein_module/models/`, `infection_module/models/`*), the raw/processed datasets and the demo images (*`data/demo/`*) are git-ignored because of their size, so they are supplied in the submission folder next to the repository. The public datasets can be re-downloaded with* `scripts/download_datasets.py` *(Section 13).*

Contents

1. Problem Statement and Objectives
2. Finalized System Architecture / Methodology
3. Module-wise Implementation
4. Algorithms, Models and Technologies Used
5. Dataset / Database Details
6. Implementation Screenshots
7. Experimental Setup
8. Testing and Test Cases
9. Results and Performance Metrics
10. Comparison and Analysis
11. Limitations
12. Conclusion
13. Instructions for Executing the Project

Appendix A. Source Code Structure

Appendix B. Dependencies and Configuration

Appendix C. Additional Evidence

1. Problem Statement and Objectives

1.1 Background / Context

Peripheral intravenous cannulation (PIVC) is among the most common invasive procedures in hospital care; more than a billion peripheral catheters are reported to be placed each year. Two problems recur. **Before insertion**, clinicians must find a usable vein by sight and touch; in patients with difficult venous access (children, the elderly, obese or dehydrated patients, darker skin) first-attempt failure is commonly reported at 30–50 %, and repeated attempts cause pain, vessel damage and treatment delay. **After insertion**, a sizeable share of catheters fail early through phlebitis, infiltration or infection, and the bedside check is a subjective visual inspection in which early, mild redness is easily missed or cannot be compared with how the site looked the day before.

1.2 Problem Statement

There is no integrated, auditable decision-support tool that (a) helps choose an insertion site before cannulation and (b) measures change at the cannula site against the patient's own baseline afterwards. Commercial vein illuminators only project an image; they do not segment veins, grade them, or suggest a site. Naive computer-vision pipelines also fail silently in this setting, and each of the following failures was observed and fixed during this project:

(1) A vein network trained on near-infrared (NIR) images produced confident vein masks on ordinary RGB phone photos, which carry no NIR vein contrast. (2) A wound-versus-skin classifier used as an infection score rated every site photo ≈ 1.00, and rated a landscape photo 0.999, because it measured tissue heterogeneity and not redness. (3) Day-to-day photos differ in lighting, white balance and camera position, which confounds any comparison across days.

IntelliCannula addresses these with refusal gates and a baseline-anchored measurement, under one rule: *the system recommends, it never acts*. It is a research prototype, not a medical device.

1.3 Project Objectives

List measurable and implementation-oriented objectives.

Objective 1 (NIR provenance gate): the vein model must never run on an RGB or unverified image: accept only frames whose SHA-256 matches the CUBITAL NIR dataset or that come from a registered NIR device with a matching token; reject phone photos (camera EXIF) and everything else with HTTP 422.

Objective 2 (vein segmentation): train an Attention U-Net (7.85 M parameters) with a weighted BCE + soft-Dice loss on subject-disjoint CUBITAL splits and reach test Dice ≥ 0.55 (achieved 0.567).

Objective 3 (insertion-site selection): from the vein mask, deterministically select a straight run (sliding-window $R^2 \geq 0.95$), measure vein width, and return an insertion point, a suggested needle angle bounded to 15°–30° and a grade; refuse when no segment qualifies.

Objective 4 (site monitoring): verify the cannula hub with YOLOv11 (confidence ≥ 0.50), verify the follow-up shows the same scene as the baseline (≥ 50 RANSAC inliers), measure redness (CIELAB Δa^*) in a skin ring against the patient's baseline, and raise advisory alerts; the change measurement must track known injected redness with $r \geq 0.99$ in a controlled validation (achieved 0.995).

1.4 Scope of the Project

In scope: NIR forearm vein segmentation and site selection; NIR provenance gate; photo-quality and cannula gates; multi-day scene registration; baseline-anchored redness measurement and alert rules; FastAPI back end with PostgreSQL/SQLite persistence and audit logging; Next.js clinician dashboard; automated tests and measured evaluation.

Out of scope: autonomous needle insertion; diagnosis of infection or phlebitis (redness is advisory and cannot confirm or exclude infection); clinical validation on patients; vein scanning with consumer RGB cameras (physically unsupported); user authentication (the prototype accepts synthetic or de-identified data only).

2. Finalized System Architecture / Methodology

2.1 Overall System Architecture

The system has four layers (Figure 1). The **clinician dashboard** (Next.js) talks to the **FastAPI server** over REST and a WebSocket alert channel. The server performs upload validation, the NIR modality gate, the photo-quality gate and audit logging, and calls two in-process ML modules whose models are loaded once and cached: the **vein module** (M1: NIR provenance → preprocessing → Attention U-Net → site selector) and the **site module** (M2/M3: quality gate → YOLOv11n cannula gate → ORB-RANSAC scene check → CIELAB redness and alert rules). Results are persisted through SQLAlchemy to PostgreSQL (automatic SQLite fallback) with images in a local or S3/MinIO store.

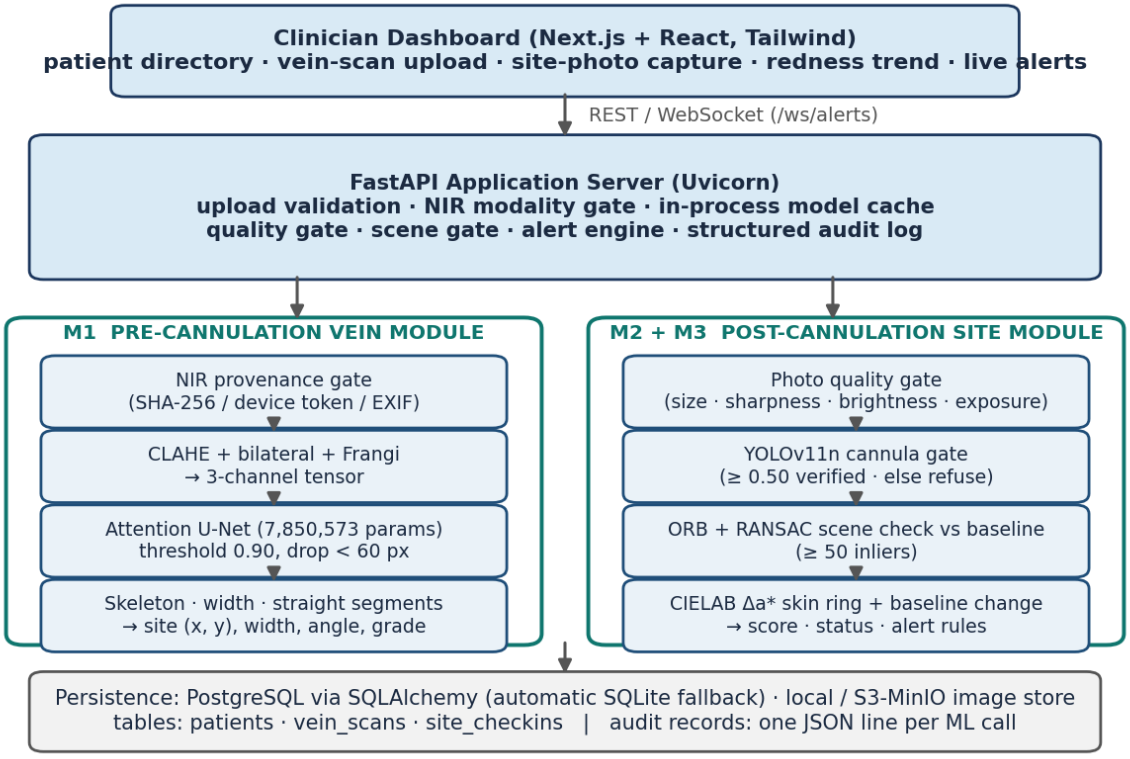


Figure 1. Finalized system architecture

2.2 End-to-End Workflow / Methodology

Before insertion a clinician uploads an NIR forearm frame. The provenance gate checks it; a verified frame is preprocessed (CLAHE, bilateral filter, Frangi vesselness), segmented, post-processed (threshold 0.90, components under 60 px removed) and passed to the site selector, which returns the insertion point, vein width, suggested angle and grade, plus an overlay. **After insertion**, a daily RGB photo of the site passes the quality gate and the cannula gate. The first accepted photo becomes the patient's baseline; each follow-up is aligned to it (ORB + RANSAC, ≥ 50 inliers), colour-matched on white/grey objects away from the site, and scored for redness in the skin ring around the hub. The score, the change since baseline and the alert rules produce a status; the result is stored, written to the audit log and pushed to the dashboard. Every gate refuses rather than guesses (red boxes in Figure 2).

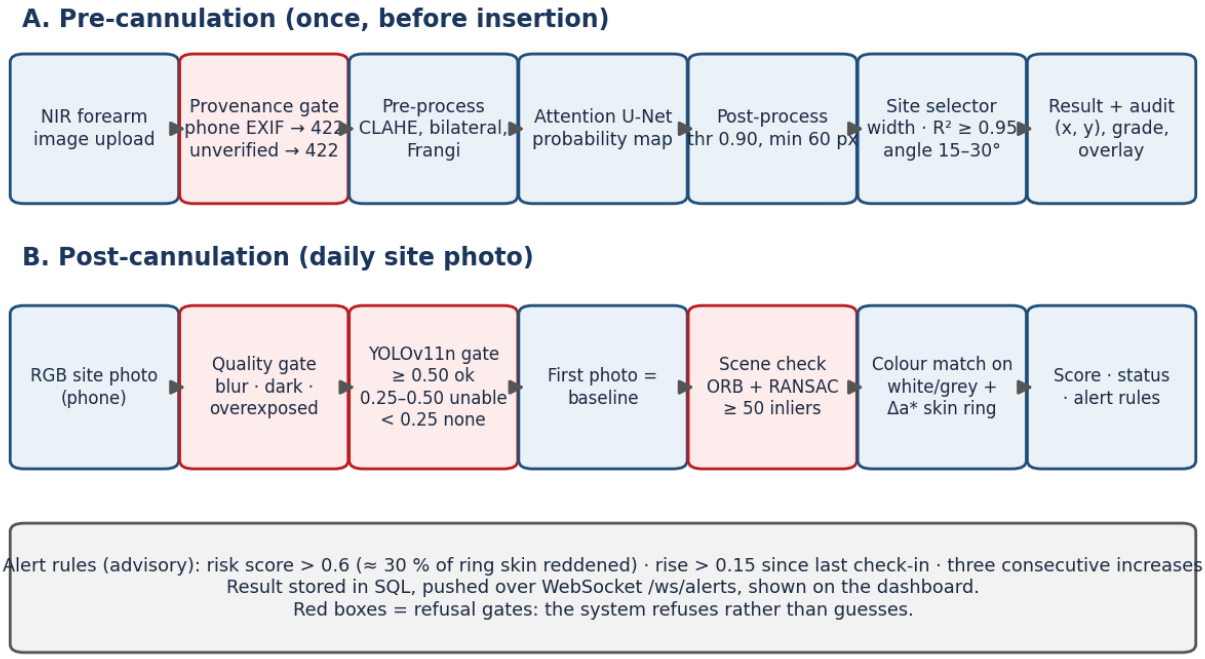


Figure 2. End-to-end project workflow / methodology

3. Module-wise Implementation

Summarize all implemented modules. Every proposed module should appear in this table.

Module No.	Module / Feature	Purpose	Implementation Status	Evidence
M1	Pre-cannulation vein guidance	NIR provenance gate, Attention U-Net segmentation, insertion-site, angle and grade	100% Complete	vein_module/, backend/app/api/vein.py, Fig 3, 8
M2	Cannula localisation and quality gate	Photo-quality screening and YOLOv11n cannula hub verification (anchors the site ROI)	100% Complete	infection_module/src/cannula_detector/, backend/app/core/ml.py, Fig 4
M3	Longitudinal redness and alert engine	Scene registration, baseline-anchored CIELAB redness measurement, status and alert rules	100% Complete	infection_module/src/erythema.py, registration.py, temporal_tracker.py, Fig 5, 11
M4	Clinician dashboard and audit API	Next.js UI, REST/WebSocket API, patient history, persistence, structured audit log	100% Complete	frontend/, backend/app/, Fig 7–10

3.1 Module 1: Pre-Cannulation Vein Guidance

Purpose and function. Find usable veins in an NIR forearm image and recommend an insertion site, with the width, suggested angle and grade behind it.

Inputs. One NIR forearm image, plus either a matching SHA-256 in the local CUBITAL registry or the headers `X-NIR-Device` / `X-NIR-Device-Token` of a registered device.

Processing. (1) *Provenance gate* (`backend/app/core/modality.py`): pixel statistics cannot separate NIR from RGB here (CUBITAL chroma spread overlaps RGB photos), so acceptance is by provenance; camera EXIF → rejected as RGB, anything else unverified → HTTP 422, and the model is never called. (2) *Preprocessing* (`vein_module/src/preprocessing.py`): CLAHE (clip 2.0, 8×8) on L of LAB with bilateral smoothing, and a polarity-filtered Frangi vesselness map (σ = 1,3,5,7,9; dark ridges only); the network input is [preprocessed R, preprocessed G, Frangi]. (3) *Segmentation*: Attention U-Net, 7,850,573 parameters, sigmoid output thresholded at 0.90, connected components under 60 px removed. (4) *Site selection* (`site_selector.py`): skeletonise, measure width by distance transform, keep runs that are straight (sliding 9-px window, $R^2 \geq 0.95$), ≥ 3 px wide and ≥ 15 px long; score = 0.5·width + 0.3·length + 0.2·(distance from edge); confidence is the margin over the runner-up (a single candidate scores 0.5); the angle is a heuristic proxy for depth, clamped to 15°–30°.

Outputs. Binary mask, overlay, insertion point (x, y), vein width (px), angle (°), grade, confidence, reason, model version, image SHA-256, inference time. Result of the recorded live run (Figure 3): site (269, 238), width 4.24 px, 25.1°, Grade C (Difficult), confidence 0.50 (single candidate), verified as a CUBITAL test-split frame.

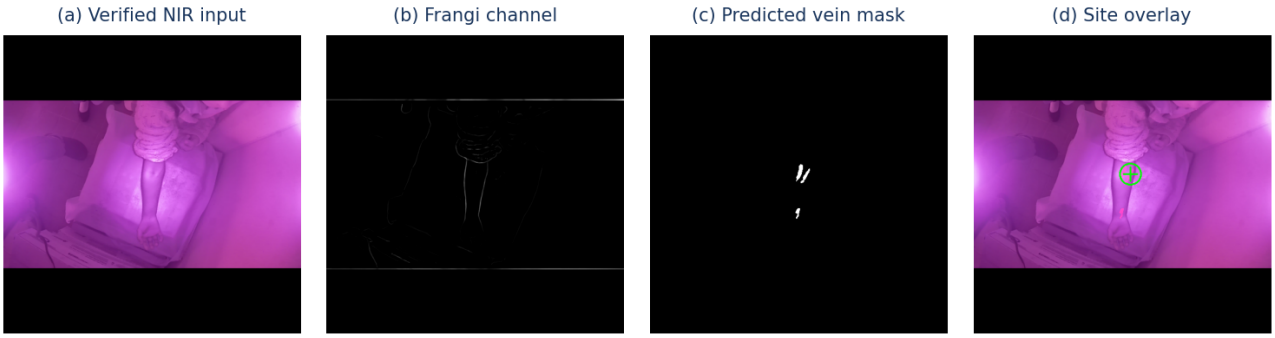


Figure 3. Module 1 implementation / output screenshot: verified NIR input, Frangi channel, predicted mask and the selected insertion site (live API output)

3.2 Module 2: Cannula Localisation and Quality Gate

Purpose and function. Make sure a site photo is usable and that it really shows a cannula, and anchor the measurement region on the hub.

Inputs. One RGB site photo (phone) with the hub in frame.

Processing. *Quality gate* (`backend/app/core/ml.py`): short side ≥ 256 px, Laplacian variance ≥ 20 (measured on ≤ 1024 px; sharp photos score ≥ 57, blurred copies ≤ 3.3), mean brightness ≥ 40, overexposed pixels ≤ 25 %; otherwise a retake is requested before any model runs. *Cannula gate*: YOLOv11n (Ultralytics) fine-tuned for 15 epochs at 640 px on 90 synthetic composites (a procedural hub on real skin backgrounds); the model is loaded once and cached in the process. Confidence ≥ 0.50 → verified; 0.25–0.50 → “unable to verify” (refused); < 0.25 → “no cannula” (refused); nothing is stored and no score is produced. On a *follow-up* whose baseline hub was verified and whose scene matches, a weak detection (0.25–0.50) is accepted and the hub position is taken from the baseline (red skin lowers detector confidence).

Outputs. Hub bounding box, detector confidence, verification state (`detected` / baseline position), or an HTTP 422 with a retake instruction. The recorded run (Figure 4) verified the hub at confidence 0.71 and refused a landscape photo (0.24).



Figure 4. Module 2 implementation / output screenshot: cannula gate accepting a hub photo and refusing a photo with no cannula (live API output)

3.3 Module 3: Longitudinal Redness and Alert Engine

Purpose and function. Measure how much redder the skin around the cannula is than at baseline, robustly to lighting and camera movement, and raise advisory alerts.

Inputs. The current photo, the patient's baseline photo and the verified hub box.

Processing. (1) *Scene check* (`registration.py`): 2,000-feature ORB matching, Lowe ratio 0.75, RANSAC homography; fewer than 50 inliers → refused as a different scene (same scene 881–1,865 inliers, unrelated ≤ 11). (2) *Skin ring* (`erythema.py`): YCrCb skin mask (grey tape, dressing and the hub are excluded), ring from 0.75 to 5.0 hub half-sizes around the hub. (3) *Redness*: a* in CIELAB; reference = median a* of the farthest 30 % of visible skin in the same photo; a pixel counts as reddened when Δa* > 6; *risk score* = reddened fraction / 0.5, capped at 1. (4) *Change since baseline*: align the follow-up to the baseline, match colours on white/grey objects outside the ring, and take the share of baseline-ring skin whose a* rose by ≥ 6. (5) *Status and alerts* (`temporal_tracker.py`): alert when the score > 0.6 (≈ 30 % of ring skin reddened), or rises by > 0.15 since the last check-in, or increases three check-ins in a row. Statuses: Baseline recorded · No significant visible change · Visible change detected (change ≥ 10 pts or level ≥ 2) · Review recommended (alert) · Unable to compare (for example no white/grey reference in frame).

Outputs. Score, reddened fraction, redness level, change since baseline, scene inliers, status, plain-language explanation, alert flag and reason, trend. In the recorded run (Figure 5) the scores were 0.11 → 0.21 → 0.82 and the third check-in raised an alert (score > 0.6 and a +0.61 jump).

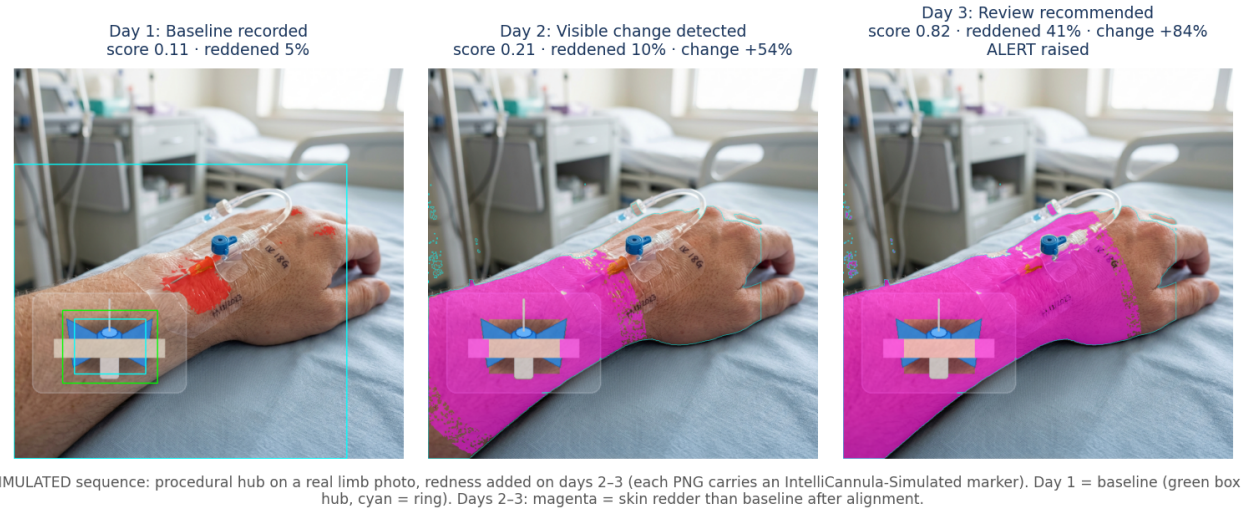


Figure 5. Module 3 implementation / output screenshot: baseline and two follow-ups of a simulated sequence with the change map and alert state (live API output)

3.4 Integration of Modules

API. The dashboard calls `POST /patients`, `POST /patients/{id}/vein-scan` (201, or 422 when not verified NIR), `POST /patients/{id}/site-check` (201, or 422 when the photo or cannula is rejected), `GET /patients/{id}/history`, `GET /modality/nir`, `GET /models`, and listens on the WebSocket `/ws/alerts`. ML endpoints return 503 with the missing package named if the ML environment is absent, never a fabricated result.

Execution flow. An upload is validated, hashed, passed through the gates, run through the cached model(s), stored through SQLAlchemy (tables `patients`, `vein_scans`, `site_checkins`; images in the local store or S3/MinIO) and recorded as one structured JSON audit line (timestamp, endpoint, patient, model version, image SHA-256, status, latency). Alerts are broadcast on the WebSocket.

M4 – dashboard. Next.js 16 / React 19 / Tailwind 4: patient directory with alert and “site check due” badges, a patient page with the NIR upload, the site-photo capture, the redness-over-time chart (alert line at 0.6) and a history table, and a model panel fed live from `/models` that states what is and is not validated. A banner states that the system is advisory.

4. Algorithms, Models and Technologies Used

Category	Algorithm / Model / Technology	Version / Configuration	Purpose in Project
Programming Language	Python; TypeScript	Python 3.12.8; TypeScript 5, Node v24.14.1	ML, API; dashboard
Framework / Library	PyTorch; torchvision	torch 2.14.0, Apple MPS	Attention U-Net training and inference
Framework / Library	OpenCV; scikit-image; SciPy; NumPy	5.0.0; 0.25.0; 1.18.1; 2.1.3	CLAHE, bilateral, Frangi, skeleton, distance transform, ORB/RANSAC, CIELAB
Algorithm / Model	Attention U-Net	7,850,573 params; weighted BCE 0.3 + Dice 0.7; AdamW 1e-4	NIR vein segmentation
Algorithm / Model	YOLOv11n (Ultralytics)	ultralytics 8.4.123; 640 px, 15 epochs; gate ≥ 0.50	Cannula hub detection
Algorithm / Model	ORB + RANSAC; CIELAB Δa*	ratio 0.75; ≥ 50 inliers; Δa* > 6	Scene check; redness measurement
Database / Storage	PostgreSQL via SQLAlchemy; SQLite fallback	postgres:16-alpine (compose); SQLAlchemy 2.0.36	Patients, scans, check-ins; images local or S3/MinIO
API / Service	FastAPI; Uvicorn; WebSocket	0.115.6; 0.34.0; REST + `/ws/alerts`	Gates, inference, persistence, alerts
Tool / Platform	Next.js; React; Tailwind CSS	16.3.1; 19.2.8; 4	Clinician dashboard
Tool / Platform	pytest; uv; Docker; GitHub Actions	pytest 8.3.4; Docker 29.1.2	Tests, environments, infrastructure, CI

4.1 Algorithm / Model Description

Attention U-Net — a 4-stage encoder–decoder whose skip connections pass through soft attention gates, so background (sheet creases, skin glare) is down-weighted before concatenation. Veins cover only about 0.6 % of a frame, so BCE alone is minimised by predicting nothing; the loss is therefore weighted BCE (*pos_weight* estimated from the masks, weight 0.3) plus soft Dice (0.7). A first run collapsed to all-background until both measures were added. Trained with AdamW (lr 1e-4, cosine decay) for 19 epochs; the epoch-16 checkpoint had the best validation Dice (0.475) and is deployed.

Frangi vesselness and CLAHE — a hand-engineered "tube-likeness" channel and local contrast enhancement give the network the prior that veins are dark elongated ridges. scikit-image's *black_ridges* flag is a no-op in 0.25, so polarity is enforced explicitly against a local background. The network is trained with this preprocessing, so inference uses it unchanged.

Geometric site selector — deterministic and inspectable: a clinician can see why a point was chosen (width, straightness, margin). It refuses when no segment is wide, long and straight enough. The angle is a documented heuristic, not a depth measurement.

YOLOv11n — a small single-stage detector that runs in-process at bedside latency. No public cannula dataset exists, so it was trained on synthetic composites; the gate therefore has three states and refuses when unsure.

ORB + RANSAC — checks that a follow-up shows the same scene as the baseline and supplies the homography used to align them; a different scene gives ≤ 11 inliers, so a threshold of 50 separates the cases. **CIELAB Δa*** — a* isolates the green–red axis, so a shadow or bruise (which mostly moves L*) is less likely to read as redness, and differences within one photo, or against an aligned and colour-matched baseline, cancel exposure and skin tone. The first measurement (difference of two in-photo references) under-measured spreading redness (r = 0.59 against injected truth) and was replaced by the baseline-anchored measurement (r = 0.995).

4.2 Key Parameters / Hyperparameters

Parameter	Value	Reason / Role
`VEIN_MASK_THRESHOLD`	0.90	Chosen on validation only; removes speckle and halo while keeping recall 0.759. 0.95 gives higher Dice (0.592) but loses 6 pts of recall, so was not adopted.
`VEIN_MIN_COMPONENT_PX`	60 px	Removes isolated false-positive blobs smaller than a credible vein.
`CLAHE clip / tile`	2.0 / 8×8	Local contrast without amplifying noise; fixed because training used it.
`Frangi σ, β`	1,3,5,7,9; 0.5	Scales cover thin to wide veins; dark-ridge polarity enforced.
`LINEARITY_R2 / WINDOW`	0.95 / 9 px	Candidate runs must be straight and accessible.
`MIN width / length`	3.0 px / 15 px	A narrower or shorter run is not a credible target at 512×512.
`Needle angle range`	15°–30°	Shallow for wide/high-contrast (superficial) veins, steeper otherwise (heuristic).
`CANNULA_CONF_THRESHOLD`	0.50 (0.25 floor)	Verified ≥ 0.50; 0.25–0.50 unable to verify; below 0.25 no cannula.
`SCENE_MIN_INLIERS`	50	Same scene 881–1,865 inliers, unrelated ≤ 11.
`RED_DELTA_A`	6.0	Δa* above the 2–3 unit just-noticeable skin colour difference; counts a pixel as reddened.
`Ring (hub half-sizes)`	0.75 – 5.0	Skin just outside the hub up to ≈ 2.5 hub widths.
`Alert rules`	> 0.6 · > +0.15 · 3 rises	Taken from the project plan; not fitted to outcome data.
`Quality gate`	256 px · 20 · 40 · 25 %	Min short side · Laplacian variance · brightness · max overexposed.

5. Dataset / Database Details

Complete this section only if the project uses a dataset, database, knowledge base, logs, sensor data or any other persistent data source.

Item	Details
Dataset / Database Name	CUBITAL NIR forearm dataset (vein module); synthetic cannula composites (YOLO gate); AZH and Medetec wound photos (background skin images and negatives); application database (patients, vein_scans, site_checkins)
Source	CUBITAL: Salcedo et al., github.com/EdwinTSalcedo/CUBITAL (public). AZH wound dataset and Medetec gallery: public downloads (scripts/download_datasets.py). Cannula composites and demo sequences: generated internally.
Size / Number of Records	CUBITAL: 2,016 NIR images (512×512) from 1,008 subjects; 285 validation and 288 test images contain an annotated vein. AZH: 930 downloaded (830 used); Medetec: scraped galleries. Cannula set: 130 images (90 train / 15 val / 25 test).
Features / Attributes	CUBITAL 3-class masks (0 background, 1 arm, 2 vein; the vein class is used); YOLO hub boxes; database fields listed in Figure 6.
Train / Validation / Test Split	CUBITAL, subject-level (seed 42): 1,412 / 302 / 302 images = 706 / 151 / 151 subjects; 0 subjects and 0 identical images shared between splits.
Data Format	PNG/JPEG images, PNG masks, YOLO .txt labels, SQL tables, JSON results
Pre-processing	Subject-level split; resize to 512×512; CLAHE + bilateral; Frangi channel; augmentation for training; images downscaled to ≤ 1024 px for site checks
Privacy / Ethics Considerations	Public, de-identified research images only; demo patients are synthetic and labelled DEMO-; simulated site images carry an embedded IntelliCannula-Simulated marker. The prototype has no authentication and must not be used with identifiable patient data.

5.1 Data Preparation / Database Schema

Pipeline. `vein_module/src/data_loader.py --prepare` splits CUBITAL by subject (each person has two arm images, so a file-level split would leak), extracts the vein class (`mask == 2`) and writes a split manifest; `scripts/data_report.py` regenerates the data report. Vein pixels are about 0.6 % of a frame, which motivates the weighted BCE + Dice loss.

Schema. `patients` (name, unique MRN) has many `vein_scans` (image and mask URLs, insertion point, angle, width, confidence, reason, modality fields, model version, image SHA-256) and many `site_checkins` (image URL, hub box, risk score, change, alert flag and reason, trend, registration flag, model version, SHA-256). Insertion fields are nullable on purpose: a refusal with a reason is a valid result. Audit records are structured JSON log lines (see `docs/audit_logging.md`), not a table.



Figure 6. Dataset sample / database schema / data flow: CUBITAL NIR training samples and the relational schema

6. Implementation Screenshots

Provide clear screenshots showing the completed implementation. Screenshots should demonstrate the major features and outputs, not only the source code editor.

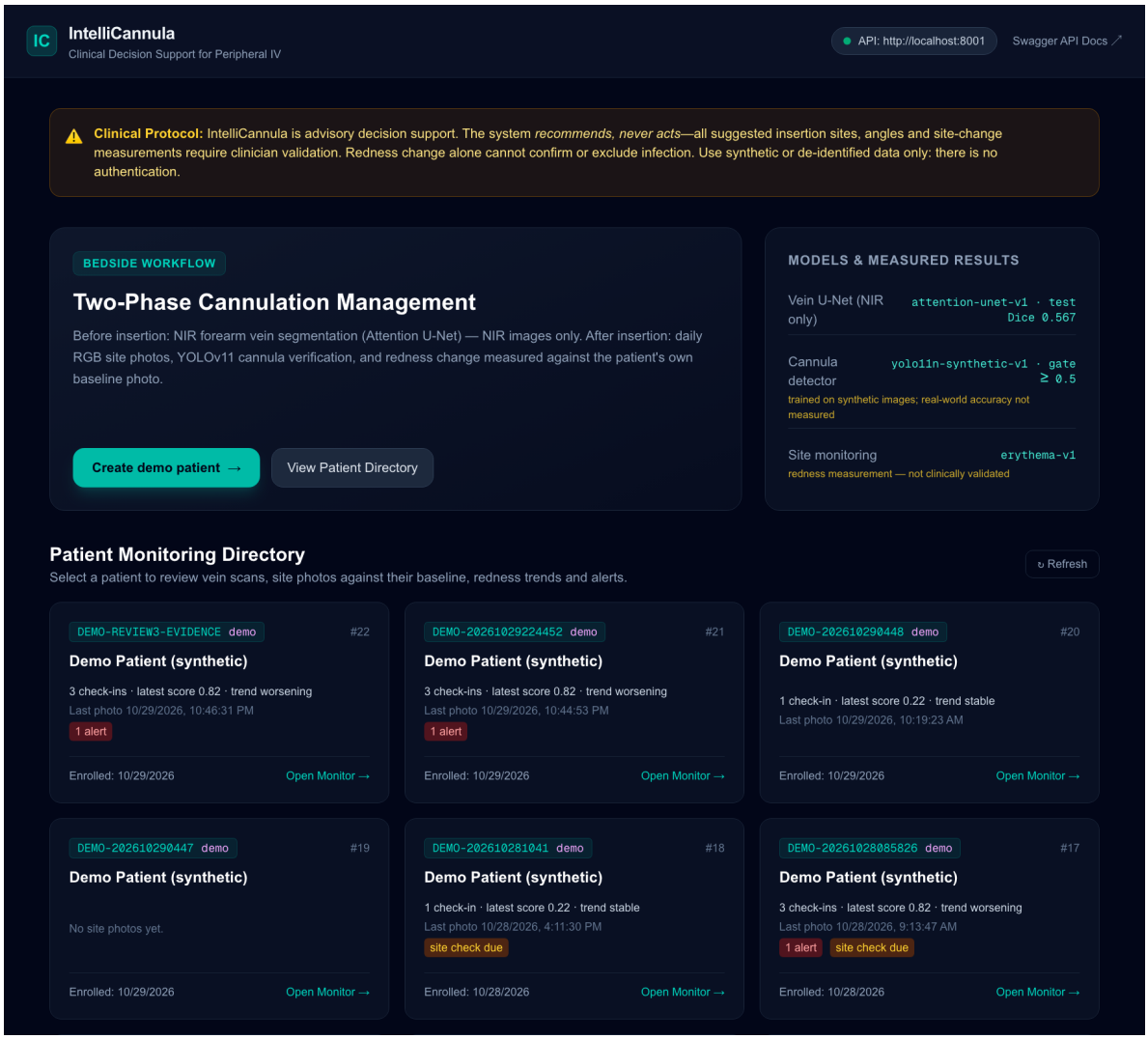


Figure 7. Clinician dashboard home page (live capture, http://localhost:3000): advisory banner, model panel and patient directory with alert badges

Shows the running Next.js dashboard connected to the API (green API indicator). The model panel is read live from `/models` and reports the vein model's test Dice 0.567, the cannula detector's synthetic-only training and the unvalidated status of redness measurement. Patient cards show check-in counts, latest score, trend, alert and "site check due" badges.

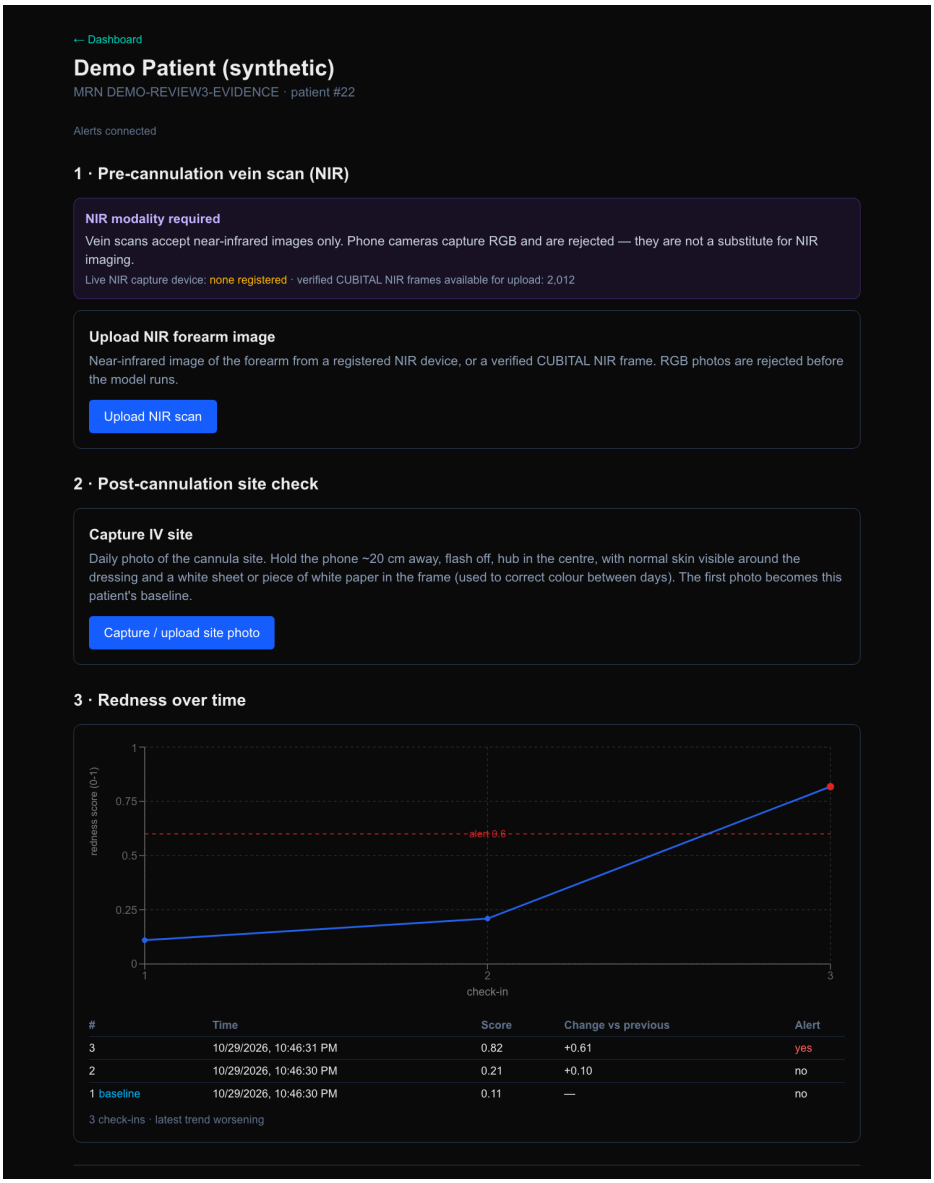


Figure 8. Patient monitoring page (live capture, patient #22): NIR-only upload notice, site capture, redness-over-time chart with the 0.6 alert line and history table

Shows the end-to-end flow after the recorded run: three check-ins at 0.11, 0.21 and 0.82, a “worsening” trend, and an alert on the third (+0.61 change versus previous). The NIR panel states that phone photos are rejected and that no live NIR device is registered; verified CUBITAL frames are available for upload.

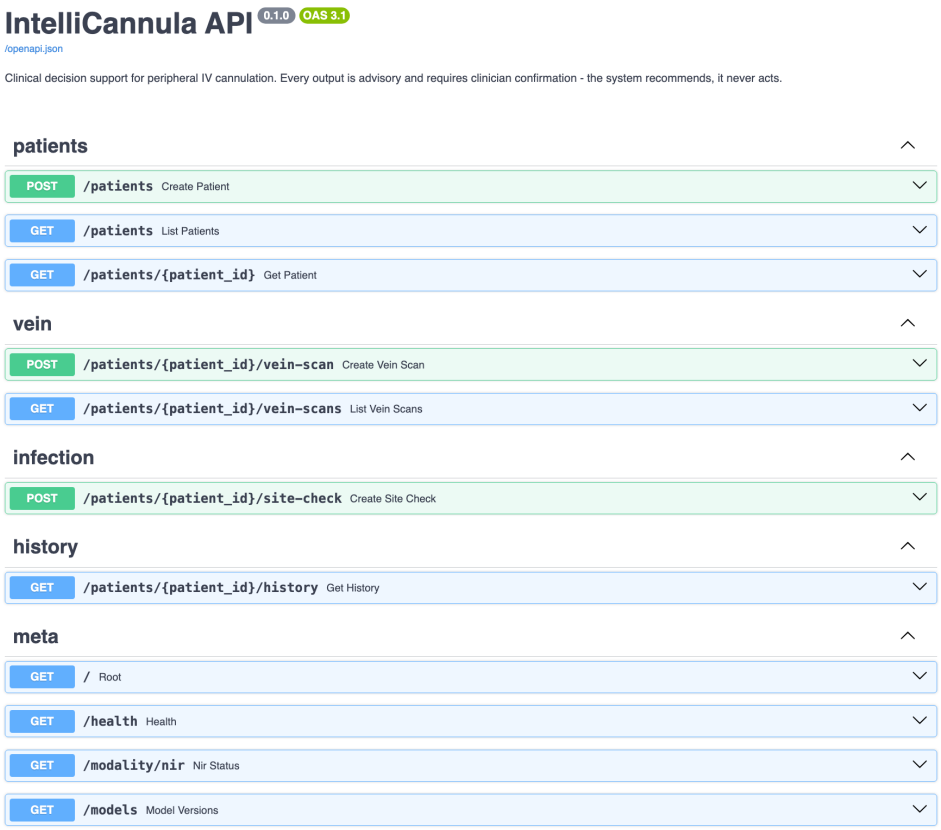


Figure 9. Interactive API documentation (live capture, http://localhost:8001/docs): patients, vein, infection, history and meta endpoints

Shows the REST surface used by the dashboard and the demo script, generated from the FastAPI application, confirming the back end is running with all routes registered.

```
$ infection_module/.venv/bin/python scripts/run_demo.py
demo patient #21 (DEMO-20261029224452)

PASS  1_vein_scan_arm.png                vein-scan  201   322 ms
      NIR frame accepted and segmented: site 269,238 grade Grade C (Difficult)
PASS  4_rgb_photo_for_vein_REJECTED.png  vein-scan  422     7 ms
      RGB photo refused for vein scan: NIR vein scan required.
PASS  SIMULATED_day1_baseline.png        site-check 201   158 ms
      baseline recorded, labelled simulated: Baseline recorded | reddened 5%
PASS  SIMULATED_day2_mild.png            site-check 201   289 ms
      compared with baseline, no alert: Visible change detected | reddened 10%,
      change +54%
PASS  SIMULATED_day3_spreading.png       site-check 201   283 ms
      change detected, alert raised: Review recommended | reddened 41%, change +84%
PASS  3_site_check_landscape_REJECTED.png  site-check 422    82 ms
      no-cannula photo refused: No cannula detected in this photo.

timeline: 3 check-ins (rejected photos create none), scores [0.11, 0.21, 0.82]

all steps passed · open http://localhost:3000/patients/21
```

Figure 10. End-to-end demo run (`scripts/run_demo.py`) against the live API: six steps, all passing, including both refusals

Shows the two modules and their gates executing against the running system: an NIR frame is accepted and segmented (322 ms); an RGB photo is refused for the vein scan (HTTP 422); a simulated baseline is recorded; a mild follow-up is compared with it; the spreading follow-up raises an alert; a no-cannula photo is refused (HTTP 422) and creates no check-in. Overlay outputs for the same run are in Figures 3–5.

7. Experimental Setup

7.1 Hardware Configuration

Component	Configuration
Processor / CPU	Apple M3, 8 cores
Memory / RAM	16 GB unified memory
GPU / Accelerator	Apple M3 integrated GPU (10 cores) via PyTorch MPS
Storage	460 GB internal SSD
Other Hardware	None attached. No live NIR imager is connected to this prototype, so the vein workflow runs on verified CUBITAL frames; site photos are RGB files/phone uploads.

7.2 Software Environment

Component	Version / Details
Operating System	macOS 26.3.1 (Apple Silicon)
Programming Language	Python 3.12.8; TypeScript 5 on Node v24.14.1
Frameworks / Libraries	PyTorch 2.14.0 (MPS), OpenCV 5.0.0, scikit-image 0.25.0, SciPy 1.18.1, NumPy 2.1.3, Ultralytics 8.4.123, FastAPI 0.115.6, SQLAlchemy 2.0.36, Next.js 16.3.1, React 19.2.8
Database / Server	SQLite fallback for the recorded runs (the Docker daemon was not running, so the API fell back automatically); PostgreSQL 16 (postgres:16-alpine) is the configured primary via docker-compose. Uvicorn 0.34.0.
IDE / Development Tools	Visual Studio Code, Claude Code, uv, pytest, Git/GitHub Actions
Deployment Environment	Local: native Python (uv) for API and ML so MPS stays available; Docker for infrastructure; optional containerised stack (`docker compose --profile full`)

7.3 Experimental / Evaluation Protocol

Vein segmentation. Subject-level split (seed 42). Training: AdamW lr 1e-4 with cosine decay, 19 epochs, best-validation checkpoint (epoch 16). Post-processing (threshold, minimum component size, TTA, hysteresis) was chosen on the validation split only (285 images with a vein). The test split (288 images with a vein) was evaluated once with the frozen configuration; error analysis on test is analysis only and changed nothing. Per-image Dice distribution, error taxonomy (using the arm class) and split-integrity checks were recorded.

Cannula gate. Synthetic val/test for the threshold; false acceptance on 400 real images without a cannula (200 CUBITAL, 100 AZH, 100 Medetec); individual real IV photos. No labelled real cannula set exists, so real-world false rejection is not estimated.

Site monitoring. Controlled validation (`scripts/validate_site_monitoring.py`) on real limb photos with known injected redness: +3 … +24 a* units, two spreads, skin only; 48 follow-ups on the 4 usable backgrounds plus lighting, white-balance, JPEG and phone-movement “nuisance” photos with true change 0. The measurement is the real API code path. Ground truth is the share of baseline-ring skin whose a* rose by ≥ 6.

Latency was measured on Apple MPS, warm and repeated, from image-in to result-out. **Tests**: pytest suites run on 29 Oct 2026 (Appendix C). All simulated images are labelled as such and none is presented as clinical evidence.

8. Testing and Test Cases

8.1 Testing Strategy

Unit tests (192 tests, `vein_module/tests` 91 and `infection_module/tests` 101): data loaders and split integrity, preprocessing, model shape and parameter count, training loop, evaluation metrics, site selector, cannula detector and pipeline, registration, erythema and temporal tracker. **API / integration tests** (68 tests, `backend/tests`): REST contract, modality gate (11 cases: phone EXIF, plain, greyscale, “NIR” file name, device without or with wrong token, corrupt and oversized files, registered device, dataset frame), detection and quality gates, site monitoring (baseline, follow-up, different scene, blur, weak detection, thresholds, UTC timestamps, patient isolation). **System test**: `scripts/run_demo.py` drives six steps through the running API and asserts each outcome. **Model validation** and **performance testing** are reported in Section 9. Frontend: `tests/timeline.test.ts`, type-check and lint (see README).

8.2 Test Case Summary

TC ID	Module / Feature	Test Input / Condition	Expected Output	Actual Output	Status
TC01	Modality gate	Phone-style RGB photo (demo file 4_rgb_photo…) sent to vein-scan	HTTP 422; model not run	HTTP 422 “NIR vein scan required … cannot be verified” in 7 ms	Pass
TC02	Modality gate / M1	CUBITAL test-split NIR frame (1_vein_scan_arm.png)	HTTP 201; verified; site or refusal with reason	201; verified (cubital_dataset, test split); site (269, 238), 4.24 px, 25.1°, Grade C, conf 0.50; 191–322 ms	Pass
TC03	Modality gate	Plain / greyscale / “NIR”-named image; device ID without or with wrong token	HTTP 422 unverified; model never called	Rejected in automated tests (test_modality.py)	Pass
TC04	Modality gate	Registered device with matching token; camera-EXIF photo	Device accepted; EXIF photo rejected as RGB	As expected in automated tests	Pass
TC05	Quality gate	Blurred, dark and overexposed site photos	Refused before scoring	Refused (test_detection_gates.py, test_site_monitoring.py)	Pass
TC06	Cannula gate	Landscape photo with no cannula	HTTP 422; nothing stored	422 “No cannula detected … retake” (82 ms); no check-in created	Pass
TC07	Cannula gate / M2	Simulated baseline photo with hub	Hub verified; baseline recorded	Confidence 0.71, 201, “Baseline recorded”, reddened 5 %, score 0.11	Pass
TC08	Scene check	Follow-up of an unrelated scene	Not compared (< 50 inliers)	Refused as a different scene (test_site_monitoring.py)	Pass
TC09	M3 / compare	Mild follow-up (day 2)	Compared with baseline; no alert	1,895 inliers; “Visible change detected”, reddened 10 %, change +54 %, score 0.21, no alert	Pass
TC10	Alert engine / M4	Spreading follow-up (day 3)	Alert raised and shown	1,866 inliers; “Review recommended”, reddened 41 %, change +84 %, score 0.82 (+0.61), alert	Pass

8.3 Defects Identified and Resolved

Issue ID	Issue / Defect	Resolution	Final Status
I01	Vein mask was blobs and speckle; 74 % of marked pixels were not vein (training stopped at epoch 6 of 25; fixed 0.5 threshold)	Resumed to epoch 16; threshold 0.9 and 60 px filter tuned on validation. Test Dice 0.370 → 0.567	Resolved
I02	A clear 13.8 px vein was graded “Not Recommended” (grade used the selector's separation margin)	Grade from vein width; refusal maps to “Not Recommended”	Resolved
I03	A landscape photo scored infection risk 0.999 (no cannula check; image centre scored)	Strict cannula gate; no centre fallback	Resolved
I04	Every site photo scored ≈ 1.00 (LightGBM trained on chronic wound vs skin proxy labels)	Replaced by measured peri-cannula redness versus the patient's baseline	Resolved
I05	RGB phone photos were accepted for vein scanning (no modality check)	NIR provenance gate (SHA-256 registry / device token; EXIF rejected); model never called on rejection	Resolved
I06	Follow-up of a different scene was compared as the same site	ORB/RANSAC scene check; < 50 inliers refused	Resolved
I07	Site check took 1.7–2.8 s (detector started in a new subprocess per request)	In-process cached detector: 42 ms	Resolved
I08	Vein forward pass 597 ms on CPU	Apple MPS: 77 ms with identical masks (40/40 test images, Dice 1.0000)	Resolved
I09	Change since baseline under-measured (r = 0.59, mean error 21.5 pts)	Pixel-wise comparison with the aligned, colour-matched baseline: r = 0.995, mean error 3.6 pts	Resolved
I10	Most-inflamed follow-ups refused (13 of 48): detector confidence drops on red skin	Weak detection accepted on a follow-up that matches a verified baseline; 0 of 48 refused	Resolved
I11	Chart times 5.5 h early; dashboard showed stale hard-coded metrics	UTC normalisation with regression tests; live model panel from /models	Resolved

9. Results and Performance Metrics

9.1 Final Results

All four objectives were achieved and demonstrated end to end on the running system. **O1:** the vein model is never called on an RGB or unverified image (11 gate tests; phone photo refused in the live run). **O2:** test Dice 0.567 (median 0.610) against the 0.55 target, up from 0.370 at the first checkpoint; no false positives off the arm beyond 0.3 % of false-positive pixels. **O3:** the selector returns a point, width, angle and grade, or refuses with a reason. **O4:** the cannula gate rejects 98–99 % of real non-cannula images; scenes are matched with a wide margin (881–1,865 inliers versus ≤ 11 against a threshold of 50); the baseline-anchored change measurement tracks injected redness with r = 0.995 and raised 0 false alarms on lighting, white-balance, compression and movement photos. These results show that the system *measures* redness change correctly under controlled conditions; they do not show that redness predicts phlebitis, which needs nurse-assessed IV-site photographs.

9.2 Performance Metrics

Metric	Result	Target / Benchmark	Interpretation
Vein Dice (test, 288 vein images)	0.567 (median 0.610; P10 0.350, P90 0.741)	≥ 0.55 (O2); first checkpoint 0.370	Target met. 17.7 % of images score below 0.4.
Vein IoU	0.413	First checkpoint 0.239	+73 % relative.
Vein precision / recall	0.495 / 0.758	First checkpoint 0.262 / 0.814	Precision nearly doubled; recall traded for fewer false positives.
False positives off the arm	0.3 % of FP pixels	n/a	47 % within 2 px of a true vein, 53 % on the arm elsewhere.
Cannula gate false acceptance (real, no cannula)	1 % CUBITAL, 1 % AZH, 2 % Medetec (n = 200/100/100)	—	A false acceptance on a follow-up is still stopped by the scene check.
Cannula gate on synthetic test	0 % false acceptance, 5 % false rejection (1/19)	—	Synthetic only. One real IV-dressing photo accepted (0.79); one real hand photo missed (0.19).
YOLOv11n mAP@50 (synthetic val)	0.995	—	Measured on synthetic composites; not evidence of real-world accuracy.
Scene check inliers	881–1,865 same scene; ≤ 11 unrelated	Threshold 50	Wide margin on simulated moves (rotation 8°, re-save).
Change measurement vs injected redness	r = 0.995; MAE 3.6 pts; 16/16 detected, 8/8 not flagged	Original method r = 0.59, MAE 21.5	With a white card in frame: r = 0.996, MAE 3.0, 24/24 and 12/12. 0 of 27 nuisance photos falsely flagged.
Vein model forward pass (MPS)	77 ms	597 ms on CPU	Identical masks on 40/40 test images.
Vein request, warm (MPS)	185 ms (live run 191–322 ms)	3.4 s first request	Includes model load and NIR-registry build on the first call.
Site check request	158–289 ms (live run)	1.7–2.8 s before caching	Detector 42 ms cached; whole check includes scene and redness.
Automated tests	260 passed (192 + 68), 0 failed	—	21.83 s + 4.48 s on 29 Oct 2026.

9.3 Result Figures / Graphs

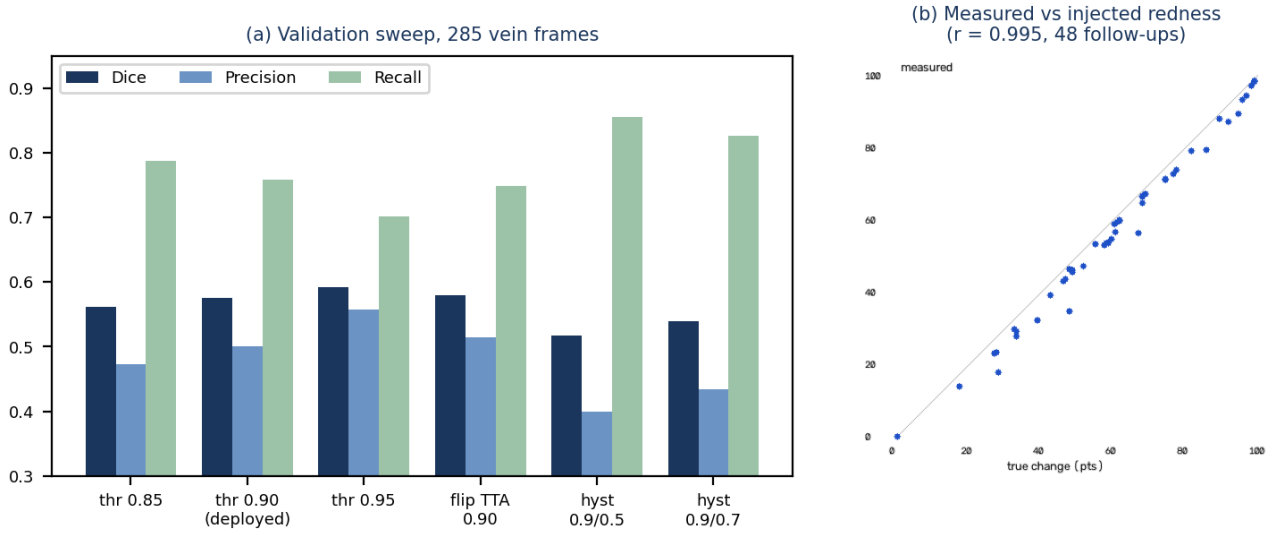


Figure 11. (a) Validation sweep of inference-time settings; (b) measured versus injected redness (r = 0.995) across lighting changes

(a) The deployed setting (threshold 0.90, 60 px filter) balances precision and recall; threshold 0.95 has the highest Dice but misses more vein, and flip-TTA adds 0.005 Dice for twice the latency, so neither was adopted. (b) Each point is one follow-up of the controlled validation: measured change tracks the injected change closely and slightly under-measures (worst case 48 % true vs 35 % measured), i.e. it errs toward missing a little change rather than inventing it.

10. Comparison and Analysis

Use this section when a baseline, existing method, previous version or competing approach is available.

Method / System	Vein Dice	Vein precision / recall	Change vs injected truth	Observation
First checkpoint (epoch 6, threshold 0.5; in-photo reference)	0.370	0.262 / 0.814	r = 0.59, MAE 21.5 pts	Speckled masks, markers on the bedsheet, change under-measured, no gates.
Threshold 0.95 (validation only, not adopted)	0.592 (val)	0.557 / 0.701	—	Higher Dice from discarding uncertain pixels; misses ~6 pts more vein.
Implemented system (epoch 16, threshold 0.90, baseline-anchored change)	**0.567**	**0.495 / 0.758**	**r = 0.995, MAE 3.6 pts**	Gated, auditable, measured; limitations in Section 11.

10.1 Analysis of Results

Strengths. Most of the gain came from resuming training to epoch 16 and from threshold 0.90 with a 60 px filter; false positives almost disappeared from the background (0.3 % off-arm). The largest improvement in the site module came from changing *what is measured*: comparing pixels with an aligned, colour-matched baseline instead of subtracting two in-photo references, because spreading redness also reddens the far skin used as the reference and cancels itself. Gates (NIR provenance, quality, cannula, scene) convert silent failures into explicit refusals.

Trade-offs and failure cases. Recall was traded for precision (0.814 → 0.758). Thin veins (< 4 px, 22.6 % of vein pixels) have recall 0.70 against 0.76 for wider ones, and veins are often fragmented (median 1.5 predicted pieces per annotated vein). High-contrast frames score lowest (Dice 0.536 versus 0.591 for low contrast); in the worst case (Dice 0.02) the annotation omits a visible vein, so some “false positives” are likely unannotated veins (a hypothesis that needs re-annotation to test). The cannula detector trained on synthetic images missed the hub on 8 of 12 validation backgrounds, including the real IV-hand photo; only 4 backgrounds could be used. Without a white or grey object away from the site, a lighting change cannot be told from redness, so such comparisons are reported as “unable to compare reliably” instead of guessed (24 of 48 follow-ups deferred as photographed; 12 of 48 with a white card).

Practical significance. The vein module is a decision aid that shows its reasoning and refuses when unsure; agreement with practitioner-marked insertion points has not yet been measured. The site module shows that redness change can be measured reproducibly on a phone photo; whether that predicts phlebitis is an open clinical question.

11. Limitations

1. Hardware: vein scanning needs NIR imaging (~850 nm); no NIR device is attached, so the workflow runs on verified CUBITAL frames and cannot use an ordinary phone camera.

2. Population: CUBITAL is 1,008 subjects aged 5–22 (median 11), 97 % recorded as slim; transfer to elderly, obese or dehydrated adults, where veins are hardest to find, is untested.

3. Site selection is unvalidated against practice: agreement with nurse-marked insertion points was not computed; the needle angle is a width/contrast heuristic, not a measured depth; in the live run the selector had a single candidate (confidence 0.50, Grade C).

4. Segmentation quality: test Dice 0.567 is moderate; 17.7 % of test images score below 0.4; thin veins and fragmented veins are the main misses.

5. Cannula detector: trained on 90 synthetic composites; real-world accuracy is unmeasured (one real hand photo was missed); it may degrade with tape, blood staining or opaque dressings.

6. No longitudinal clinical data: site sequences are simulated; alert thresholds (0.6, +0.15, three rises) come from the project plan and their false-alarm and miss rates are unmeasured; redness change cannot confirm or exclude infection.

7. Validation scope: controlled validation used 4 mostly light-skin backgrounds, smooth injected redness and a white/grey reference; darker skin and streaking phlebitis are not covered.

8. Deployment: the prototype has no authentication or clinical safeguards beyond the advisory design; synthetic or de-identified data only; research prototype, not a medical device.

12. Conclusion

IntelliCannula implements both modules of the proposed system and integrates them behind one API and dashboard. The pre-cannulation module verifies NIR provenance, segments veins with an Attention U-Net (test Dice 0.567, median 0.610; 77 ms forward pass on MPS) and selects an insertion site with width, angle and grade, refusing when unsure. The post-cannulation module gates photo quality, verifies the cannula hub, checks scene continuity (≥ 50 inliers) and measures baseline-anchored CIELAB redness change (r = 0.995 against injected truth) with advisory alerts. 260 automated tests pass and the full workflow runs end to end against the live system, including the refusal paths.

All four project objectives are met and the Review 3 implementation is complete. The contribution is a working, auditable decision-support pipeline whose limits are stated; clinical validation (adult cohorts, nurse-marked insertion points, longitudinal IV-site photographs with phlebitis grading, real NIR hardware) is the next step and is not claimed here.

13. Instructions for Executing the Project

13.1 Prerequisites

macOS (Apple Silicon recommended for MPS) or Linux; Python 3.12 with `uv`; Node.js 20 or newer (tested on v24); Docker (optional, PostgreSQL); `brew install libomp` on macOS (LightGBM, imported by the legacy risk module). Trained checkpoints (`vein_module/models/attention_unet_best.pt`, `infection_module/models/cannula_yolo11.pt`) and `data/demo/` are supplied in the submission folder (git-ignored). Optional environment variables: `DATABASE_URL` (SQLite is used automatically when PostgreSQL is unreachable), `NIR_DEVICES`, `ML_DEVICE`.

13.2 Installation / Setup Steps

Step 1: `cd intellicannula && cp .env.example .env`

Step 2: `uv venv infection_module/.venv --python 3.12`

Step 3: `uv pip install --python infection_module/.venv -r infection_module/requirements.txt -r vein_module/requirements.txt`

Step 4: `uv pip install --python infection_module/.venv -r backend/requirements.txt`

Step 5: `cd frontend && npm install` *(optional:* `docker compose up -d` *for PostgreSQL)*

13.3 Execution Steps

Step 1: `cd backend && ../infection_module/.venv/bin/python -m uvicorn app.main:app --host 0.0.0.0 --port 8001` → *http://localhost:8001/health*

Step 2: `cd frontend && npm run dev -- -H 0.0.0.0 -p 3000` → *http://localhost:3000*

Step 3: `infection_module/.venv/bin/python scripts/build_demo_set.py` *(writes data/demo/)*

Step 4: `infection_module/.venv/bin/python scripts/run_demo.py` *(API running; prints PASS for six steps)*

Step 5: `infection_module/.venv/bin/python -m pytest -q` *and* `cd backend && ../infection_module/.venv/bin/python -m pytest -q`

13.4 Sample Input and Expected Output

Item	Example / Description
Sample Input	data/demo/1_vein_scan_arm.png (CUBITAL test-split NIR frame); data/demo/SIMULATED_day1…3 site photos
Command / Action	`python scripts/run_demo.py`, or upload the files from the patient page at http://localhost:3000/patients/<id>, or `POST /patients/{id}/vein-scan` with the file
Expected Output	Vein scan 201: verified, site (269, 238), width 4.24 px, 25.1°, Grade C; RGB photo 422; site checks 201 with scores 0.11 → 0.21 → 0.82 and an alert on the third; no-cannula photo 422; “all steps passed”

13.5 Troubleshooting

Problem	Possible Cause	Resolution
HTTP 422 “NIR vein scan required”	RGB/phone photo or an unverified image sent to vein-scan	Upload a verified CUBITAL NIR frame, or register a device with `NIR_DEVICES` and send both headers
HTTP 503 naming a missing package	API started from an environment without torch/OpenCV	Start Uvicorn with `infection_module/.venv/bin/python`
HTTP 422 “No cannula detected / unable to verify”	Hub not clearly in frame, or detector below 0.50	Retake with the hub and dressing centred, ~20 cm away, with a white sheet in frame
“Unable to compare reliably”	No white/grey reference away from the site, or different scene (< 50 inliers)	Include a white sheet or paper; photograph the same site from a similar view
Database connection warning	PostgreSQL not running	Automatic SQLite fallback; or `docker compose up -d`

Appendix A. Source Code Structure

Provide the folder/file structure and a short description of important files.

Path / File	Purpose
`intellicannula/`	Project root
`backend/app/main.py, api/, core/, models/, schemas/`	FastAPI app: routes (patients, vein, infection, history, alerts), settings, DB engine, ML bridge, audit log, ORM models
`backend/app/core/modality.py`	NIR provenance gate (SHA-256 registry, device token, EXIF rejection)
`backend/app/core/ml.py`	Model cache, quality gate, vein and site-check pipelines
`vein_module/src/`	data_loader, preprocessing (CLAHE, Frangi), model (Attention U-Net), train, evaluate, postprocess, site_selector, error_analysis
`infection_module/src/`	cannula_detector/, erythema.py, registration.py, temporal_tracker.py (alert rules), legacy risk_model.py
`frontend/app, components, lib`	Next.js dashboard: patient directory, patient page, charts, API client

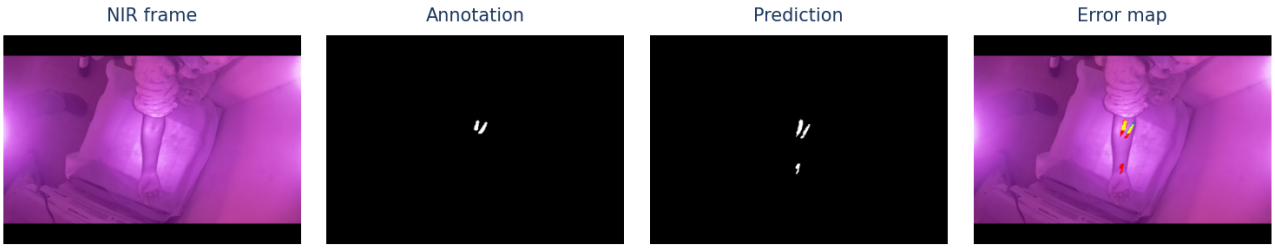
Path / File	Purpose
`data/`	raw/, processed/ and demo/ inputs and split manifests (git-ignored; supplied separately)
`vein_module/models/`, `infection_module/models/`	Trained checkpoints (Attention U-Net, YOLOv11n cannula detector)
`backend/tests, vein_module/tests, infection_module/tests, frontend/tests`	Automated tests (260 Python tests)
`scripts/`	download_datasets, build_demo_set, run_demo, validate_site_monitoring, benchmark_latency, data_report, full_system_eval, report generators
`docs/`	Measured reports: vein_error_analysis, REVIEW2_RESULTS, site_monitoring_validation, cannula_gate_eval.json, audit_logging, data_report
`docker-compose.yml, .env.example, .github/workflows/`	Infrastructure, configuration template, CI
`requirements.txt (backend/, vein_module/, infection_module/)`	Dependencies
`README.md`	Setup and execution guide

Appendix B. Dependencies and Configuration

Dependency / Tool	Version	Installation / Configuration Note
Python / uv	3.12.8	`uv venv … --python 3.12`
torch (+ MPS)	2.14.0	In infection_module/.venv; `ML_DEVICE=auto` selects MPS/CUDA/CPU
opencv / scikit-image / scipy / numpy	5.0.0 / 0.25.0 / 1.18.1 / 2.1.3	Preprocessing, vesselness, registration, colour
ultralytics	8.4.123	YOLOv11n inference and training
fastapi / uvicorn / sqlalchemy	0.115.6 / 0.34.0 / 2.0.36	`backend/requirements.txt` (also psycopg2, boto3, pydantic 2.10.4, websockets 14.1)
Next.js / React / Tailwind	16.3.1 / 19.2.8 / 4	`cd frontend && npm install`
PostgreSQL	16 (postgres:16-alpine)	`docker compose up -d`; `DATABASE_URL` in .env
Thresholds	VEIN_MASK_THRESHOLD 0.9; VEIN_MIN_COMPONENT_PX 60; CANNULA_CONF_THRESHOLD 0.5; SCENE_MIN_INLIERS 50	Environment variables, `backend/app/core/config.py`

Appendix C. Additional Evidence

C1 – Error analysis of the deployed vein model (median test frame; `docs/vein_error_analysis.md`). **C2 – Controlled site-change validation** examples (`docs/site_monitoring_validation.md`). **C3 – Automated test run** of 29 Oct 2026. Further raw evidence: `docs/vein_postprocess_experiments.json`, `docs/vein_error_analysis.json`, `docs/cannula_gate_eval.json`, `docs/site_monitoring_validation.json`, `docs/latency_benchmark.json`.



Median test frame. Yellow = correct vein, red = false positive, blue = missed vein.

Figure C1. Vein segmentation error breakdown on the median test frame

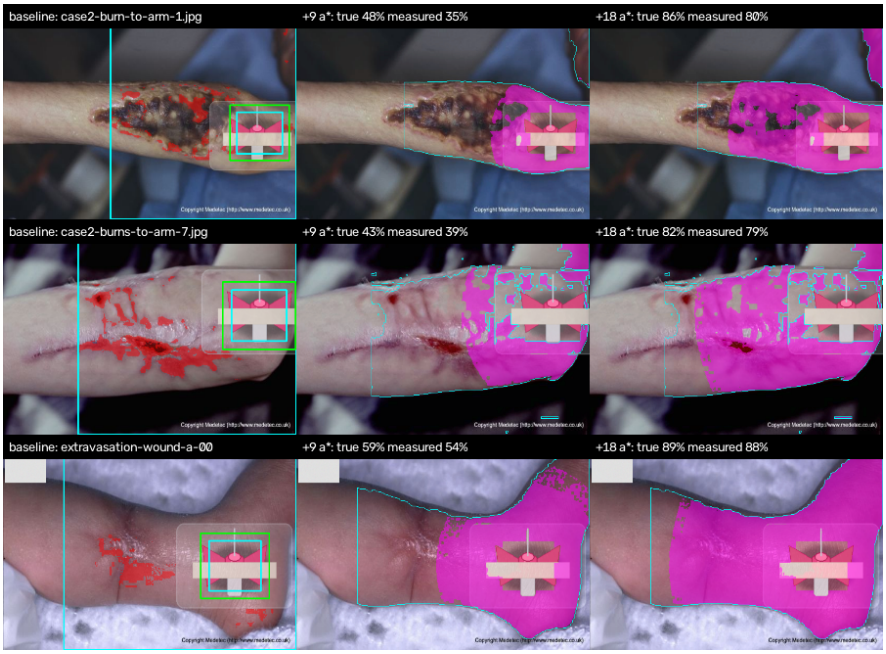


Figure C2. Controlled site-change validation: baseline (left) and change since baseline at +9 and +18 a (magenta = redder than baseline; pre-existing wounds are correctly not counted). Labels give true vs measured change.*

```
$ infection_module/.venv/bin/python -m pytest -q         # vein_module/tests + infection_module/tests
........................................................................ [ 37%]
........................................................................ [ 75%]
.............................................                            [100%]
192 passed in 21.83s

$ cd backend && ../infection_module/.venv/bin/python -m pytest -q
68 passed, 15 warnings in 4.48s
```

Figure C3. Automated test results (29 Oct 2026)

Student / Team Declaration

We confirm that the implementation documented in this report represents the completed Review 3 version of the project and that the submitted code corresponds to the demonstrated system.

Name	Registration No.	Signature	Date
Sastika M	23MIA1047		29-10-2026
Akshaya GK	23MIA1127		29-10-2026